Algorithmic Time-Series Segmentation and Condition Analysis Using Water Pump RUL Dataset

John Teetz

CMPSC 463

Table of Contents:

Project Overview.....Page 3

Installation and Usage Instructions.....Pages 3-4

Code Structure Explanation.....Page 4

Algorithmic Descriptions.....Pages 4-5

Toy Example Verification.....Page 5

Dataset Description.....Page 6

Execution Results.....Pages 6-8

Challenges and Limitations.....Page 8

Discussions and Conclusions.....Page 9

1. Project Overview

Predictive maintenance systems attempt to detect machine degradation before failure occurs. In industrial environments, machines generate large volumes of sensor data over time. Analyzing these time-series signals can help identify patterns that indicate deteriorating machine conditions.

This project analyzes a real-world water pump predictive maintenance dataset using classical algorithmic techniques rather than machine learning libraries. The goal is to characterize machine behavior using three algorithmic approaches:

1. Divide-and-Conquer Time-Series Segmentation
2. Recursive Top-Down Clustering
3. Maximum Subarray Analysis (Kadane's Algorithm)

The dataset provides timestamped sensor readings and a Remaining Useful Life (RUL) value for each observation. RUL represents the estimated number of hours remaining before the machine requires maintenance or failure occurs. To evaluate algorithmic results, the continuous RUL variable is transformed into four machine-health categories. These categories allow the outputs of segmentation, clustering, and deviation analysis to be compared with machine condition states. Unlike typical predictive maintenance research, this project intentionally avoids machine learning frameworks. Instead, it focuses on algorithm design, implementation, and interpretation of results.

2. Installation and Usage Instructions

The project is implemented in Python and requires several common libraries for data handling and visualization.

Required libraries include pandas, numpy, matplotlib, and kagglehub.

- The kagglehub library is used to automatically download the dataset from Kaggle so that the project can run without manually downloading files.
- To install the required packages, run the following command in the terminal:

```
pip install kagglehub pandas numpy matplotlib
```

- After installing the packages, the project can be executed by running the main script.

```
python main.py
```

When the program runs, it performs the following steps automatically:

1. Downloads the dataset from Kaggle.
2. Loads the first 10,000 observations.

3. Detects all sensor columns in the dataset.
4. Converts the RUL variable into four machine-health categories.
5. Performs recursive segmentation on selected sensors.
6. Executes top-down clustering on the sensor dataset.
7. Runs Kadane's algorithm on each sensor signal.
8. Saves figures and result tables to the results directory.

The program outputs segmentation plots and summary tables that are used for analysis in the report.

3. Code Structure Explanation

The code is organized into multiple modules so that each part of the analysis is clearly separated and easy to understand. The load_data module is responsible for downloading the dataset using the kagglehub package and loading the first 10,000 rows into a dataframe. This module also automatically detects all sensor columns in the dataset. The preprocess module converts the continuous Remaining Useful Life (RUL) variable into four machine-health categories using quantile thresholds. This transformation allows the algorithms to be compared against machine condition states.

The segmentation module implements the divide-and-conquer time-series segmentation algorithm. The algorithm recursively splits signals based on variance thresholds and produces stable segments. The clustering module implements a top-down clustering algorithm from scratch. The algorithm repeatedly splits clusters until four clusters are produced. The kadane_analysis module implements Kadane's algorithm to detect intervals of maximum sustained deviation in sensor signals. The visualization module generates plots that show segmentation boundaries for each analyzed sensor. The main script coordinates the entire workflow and executes all project tasks sequentially.

4. Algorithmic Descriptions

Divide-and-Conquer Segmentation

Time-series segmentation divides a signal into regions that exhibit consistent statistical behavior. In this project, segmentation is performed using a recursive divide-and-conquer approach. The algorithm first computes the variance of a signal segment. If the variance exceeds a specified threshold, the segment is divided into two halves. The algorithm then recursively applies the same process to each half of the signal. If the variance of a segment falls below the threshold, the segment is considered stable and the recursion stops. The total number of resulting segments is defined as the segmentation complexity score. A higher complexity score indicates that the sensor signal contains greater variability or more frequent changes over time.

Divide-and-Conquer Clustering

Clustering is used to group similar observations based on sensor measurements. In this project, clustering is implemented using a top-down recursive strategy. The algorithm begins with all observations placed in a single cluster. The cluster with the largest variance is selected for splitting. Two distant seed points are identified within the cluster, and each observation is assigned to the seed that it is closest to. This process divides the cluster into two smaller clusters. The algorithm continues splitting clusters until the desired number of clusters is reached. For this project, the clustering process produces four clusters. Each cluster is then compared with the machine-health categories derived from the RUL variable.

Maximum Subarray Analysis (Kadane's Algorithm)

Kadane's algorithm is used to identify the contiguous subarray with the maximum sum within a sequence. In this project, the algorithm is applied to detect intervals of sustained deviation in sensor signals. Before applying Kadane's algorithm, each sensor signal is preprocessed by computing the absolute first difference between consecutive measurements. This measures how much the signal changes between time steps. The average difference is then subtracted from each value so that the resulting sequence represents deviations from normal signal behavior. Kadane's algorithm is then applied to find the interval with the largest cumulative deviation. This interval represents a period during which the sensor signal exhibits unusually strong fluctuations. The RUL category distribution within this interval is examined to determine whether these deviations occur during healthy or degraded machine states.

5. Toy Example Verification

Before applying the algorithms to the real dataset, small toy examples were used to verify the correctness of the implementations. For the segmentation algorithm, a simple signal containing three constant regions was used. The algorithm correctly divided the signal into stable segments corresponding to the different value ranges. For clustering, a small dataset consisting of two groups of points was used. The clustering algorithm successfully separated the points into two clusters based on spatial distance. For Kadane's algorithm, a small numeric array containing both positive and negative values was tested. The algorithm correctly identified the subarray with the maximum sum. These toy examples confirm that the algorithms behave as expected before being applied to the larger dataset.

6. Dataset Description

The dataset used in this project is the Water Pump RUL Predictive Maintenance dataset obtained from Kaggle. The dataset contains time-series measurements collected from a water pump machine. Each observation includes a timestamp, multiple sensor measurements, and a

Remaining Useful Life (RUL) value. The dataset includes 52 sensor variables labeled sensor_00 through sensor_51. These sensors measure different physical characteristics of the machine, such as vibration, temperature, pressure, or other operational signals. The RUL variable represents the estimated number of hours remaining before the machine reaches failure or requires maintenance. For computational efficiency, the analysis in this project uses the first 10,000 observations from the dataset. Each row therefore represents a snapshot of the machine's operating state at a particular time.

The continuous Remaining Useful Life variable is converted into four categorical machine-health states using quantile thresholds.

First, the following quantiles are calculated:

Q10 represents the 10th percentile of the RUL distribution.

Q50 represents the 50th percentile (median) of the RUL distribution.

Q90 represents the 90th percentile of the RUL distribution.

Based on these thresholds, each observation is assigned to one of four categories. **Extremely Low RUL** represents observations where the RUL value is less than Q10. **Moderately Low RUL** represents observations where the RUL value is greater than or equal to Q10 but less than Q50. **Moderately High RUL** represents observations where the RUL value is greater than or equal to Q50 but less than Q90. **Extremely High RUL** represents observations where the RUL value is greater than or equal to Q90. These categories provide a structured way to evaluate whether algorithm outputs correspond to machine degradation states.

7. Execution Results

Task 1: Segmentation

Ten sensors were selected for segmentation analysis. The divide-and-conquer segmentation algorithm was applied to each sensor signal. The algorithm recursively divided the signals into segments until stable regions were identified. The segmentation complexity score was calculated as the total number of segments produced for each sensor. Sensors with higher segmentation complexity scores exhibited more frequent signal fluctuations and therefore more dynamic behavior over time. Segmentation plots illustrate how the algorithm detects boundaries between stable and unstable signal regions.

Below is a summary of the segmentation results from the task1_segmentation_summary.csv:

	sensor	num_segments	threshold
1	sensor_00	15	0.0006493195842282755
2	sensor_05	88	11.949091537091217
3	sensor_10	224	4.556208999407653
4	sensor_16	28	23.395475458753985
5	sensor_21	15	17.131817183272524
6	sensor_26	34	50.01203486256709
7	sensor_31	25	3916.0871827408537
8	sensor_36	8	23769.33096412996
9	sensor_41	33	6.292028993693757
10	sensor_46	26	119.05222648781718

Task 2: Clustering

The top-down clustering algorithm was applied to all 10,000 observations using the sensor measurements as input features. The algorithm recursively split clusters until four clusters were produced. For each cluster, the majority machine-health category was calculated based on the RUL class labels.

	cluster_id	size	majority_label	majority_count	label_counts
1	0	77	Moderately High RUL	39	{'Extremely High RUL': 1, 'Moderately High RUL': 39, '…
2	1	12	Moderately Low RUL	10	{'Moderately High RUL': 2, 'Moderately Low RUL': 10}
3	2	52	Moderately Low RUL	47	{'Moderately High RUL': 2, 'Moderately Low RUL': 47, '…
4	3	9859	Moderately High RUL	3957	{'Extremely High RUL': 999, 'Moderately High RUL': 395…

- Clusters dominated by extremely high RUL values likely correspond to healthy machine operating conditions, while clusters dominated by extremely low RUL values may correspond to degraded or failing states.

Task 3: Maximum Deviation Analysis

Kadane's algorithm was applied to each sensor signal to identify intervals of sustained deviation. For each sensor, the algorithm returned the starting index, ending index, and total deviation score for the maximum deviation interval. The RUL category distribution within this interval was then analyzed. Sensors whose deviation intervals occur primarily during low RUL conditions may serve as potential indicators of machine degradation.

	sensor	max_sum	start	end	dominant_rul_class	rul_class_counts
1	sensor_00	9.493931137414075	62	4831	Moderately High RUL	{'Extremely High RUL': 938, 'Moderately High RUL': 38…
2	sensor_01	184.00498048484283	1	2039	Moderately High RUL	{'Extremely High RUL': 999, 'Moderately High RUL': 10…
3	sensor_02	19.57380504361334	8990	9826	Extremely Low RUL	{'Moderately Low RUL': 10, 'Extremely Low RUL': 827}
4	sensor_03	32.33858125479592	5217	9489	Moderately Low RUL	{'Moderately Low RUL': 3783, 'Extremely Low RUL': 490}
5	sensor_04	1042.733489448872	736	5915	Moderately High RUL	{'Extremely High RUL': 264, 'Moderately High RUL': 40…
6	sensor_05	859.8426874937431	102	4790	Moderately High RUL	{'Extremely High RUL': 898, 'Moderately High RUL': 37…
7	sensor_06	7.656197497749996	2689	4489	Moderately High RUL	{'Moderately High RUL': 1801}
8	sensor_07	11.365207201663512	3884	9997	Moderately Low RUL	{'Moderately High RUL': 1116, 'Moderately Low RUL': 4…
9	sensor_08	24.514905962596192	5767	8750	Moderately Low RUL	{'Moderately Low RUL': 2984}
10	sensor_09	61.46957489649294	5832	5920	Moderately Low RUL	{'Moderately Low RUL': 89}
11	sensor_10	313.190081906402	2786	9995	Moderately Low RUL	{'Moderately High RUL': 2214, 'Moderately Low RUL': 4…
12	sensor_11	97.06918848647521	1835	7863	Moderately High RUL	{'Moderately High RUL': 3165, 'Moderately Low RUL': 2…
13	sensor_12	108.9272263430349	1870	2960	Moderately High RUL	{'Moderately High RUL': 1091}
14	sensor_13	1024.1241415446718	5857	9975	Moderately Low RUL	{'Moderately Low RUL': 3143, 'Extremely Low RUL': 976}
15	sensor_14	354.0736648364777	8622	9995	Extremely Low RUL	{'Moderately Low RUL': 378, 'Extremely Low RUL': 996}
16	sensor_16	927.4996522752425	2018	4015	Moderately High RUL	{'Moderately High RUL': 1998}
17	sensor_17	4372.12616625666	7630	9999	Moderately Low RUL	{'Moderately Low RUL': 1370, 'Extremely Low RUL': 100…
18	sensor_18	46.31404588628894	7904	9995	Moderately Low RUL	{'Moderately Low RUL': 1096, 'Extremely Low RUL': 996}

(This is a partial segment of the kadane_summary table, to see the full table, run the code and look in the results folder under tables to see the full table.)

8. Challenges and Limitations

Several challenges were encountered during the implementation of this project. One challenge involved selecting an appropriate variance threshold for the segmentation algorithm. If the threshold is too low, the algorithm produces too many segments, while a high threshold may overlook important signal changes. Another challenge involved handling missing or inconsistent sensor values within the dataset. Data cleaning and preprocessing were required to ensure that all sensor columns contained valid numeric values. Additionally, clustering results do not perfectly correspond to RUL categories because clustering is an unsupervised method that does not directly use the RUL labels during the grouping process.

Although the project demonstrates useful algorithmic approaches for analyzing machine data, several limitations exist. The segmentation algorithm relies solely on variance as the splitting criterion, which may not capture more subtle signal patterns. The clustering algorithm uses simple distance-based splitting and may not produce the most optimal cluster boundaries. Kadane's algorithm detects strong deviations but does not account for long-term signal trends or correlations between multiple sensors. Future improvements could include applying signal smoothing techniques, developing adaptive segmentation thresholds, or combining multiple sensors into composite indicators. These improvements may provide a more detailed understanding of machine degradation patterns.

9. Discussions and Conclusions

The results of this project demonstrate that classical algorithmic techniques can provide valuable insights into machine behavior using time-series sensor data. The segmentation

algorithm identified regions of stable and unstable signal behavior across sensors. Sensors with high segmentation complexity scores displayed more dynamic signal patterns. The clustering algorithm grouped observations into clusters that partially align with machine-health categories derived from the RUL variable. This suggests that sensor measurements contain meaningful patterns related to machine degradation. Kadane's algorithm revealed intervals of strong signal deviation for many sensors. In several cases, these deviations occurred during periods associated with lower RUL values, suggesting that such signals may indicate early signs of machine deterioration. Overall, the combination of segmentation, clustering, and deviation detection provides multiple perspectives for understanding machine health.